

Reviewer Pack — Minimal Rank of Tropicalized Geometric Quantization vs Branc...

Entry c1ce4a4a2e2e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 23:16:24 UTC

Minimal Rank of Tropicalized Geometric Quantization vs Branching Program Width for IP_2

Entry ID: c1ce4a4a2e2e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 23:16:24 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization Theory **Field B** (complexity object): Complexity Theory: Branching Program Complexity

Statement:

For any read-twice branching program P computing inner product mod 2, the minimal rank of its tropicalized geometric quantization is upper-bounded by $O(\log(\text{size}(P)))$ but lower-bounded by $\Omega(n^{1/4})$ for some n .

Rationale (proposer's reasoning):

Geometric quantization is a bridge between geometry and quantum mechanics that could potentially expose hidden structures in complexity theory. If the rank of the tropicalized geometric quantization is sensitive to the size of branching programs, it might reveal a new resource or tool for proving lower bounds on branching program width for IP_2.

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 725e1b8b1dec26c8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a read-twice branching program P , its tropicalized geometric quantization rank is considered supported if it meets both: (1) The upper bound condition $|\log(\text{size}(P)) - \text{rank}(P)| \leq 3$ and (2) The lower bound condition $\text{rank}(P) \geq n^{1/4}$, where $\text{size}(P)$ is the number of nodes in P and n is the input length. It is considered falsified if either condition is not met by any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropicalized geometric quantization AND branching program complexity - geometric quantization rank AND IP_2 branching program - tropicalization AND inner product mod 2 lower bound

Top relevant hits considered: - [http://arxiv.org/abs/1807.06283v2] Tropical Fano Schemes - [http://arxiv.org/abs/2601.23259v1] Geometric Quantization by Paths, Part III: The Metaplectic Anomaly - [http://arxiv.org/abs/math/0511713v2] Tropical Plane Geometric Constructions: a Transfer Technique in Tropical Geometry - [http://arxiv.org/abs/math-ph/0011018v1] Geometric Quantization on the Super-Disc - [http://arxiv.org/abs/1808.04002v3] Shifting operators in geometric quantization - [http://arxiv.org/abs/0812.1616v1] Program for calculating bounds on the minimum rank of a graph using Sage - [http://arxiv.org/abs/0705.1732v2] Fibers of tropicalization - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_branching_program(n):
        nodes = [[random.choice([0, 1]) for _ in range(2)] for _ in range(n)]
        edges = []
        for i in range(n-1):
            edges.append((i, i+1))
        return nodes, edges

    def tropicalize(mat):
        n = len(mat)
        for i in range(n):
            for j in range(n):
                if mat[i][j] == 0:
                    mat[i][j] = float('-inf')
        return mat

    def rank(matrix):
        m, n = len(matrix), len(matrix[0])
        augmented_matrix = [row + [1] for row in matrix]
        for i in range(m):
```

```

        if augmented_matrix[i][i] == 0:
            for j in range(i+1, m):
                if augmented_matrix[j][i] != 0:
                    augmented_matrix[i], augmented_matrix[j] = augmented_matrix[j], augmented_matrix[i]
                    break
            else:
                return float('inf')
        pivot = augmented_matrix[i][i]
        for j in range(n+1):
            augmented_matrix[i][j] /= pivot
        for j in range(m):
            if j != i and augmented_matrix[j][i] != 0:
                factor = augmented_matrix[j][i]
                for k in range(n+1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
        return sum(1 for row in augmented_matrix if row[-1] != 0)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    nodes, edges = generate_branching_program(n)
    size_P = len(nodes)
    tropicalized_matrix = [[nodes[i][j] ^ nodes[j][i] for j in range(size_P)] for i in range(size_P)]
    rank_value = rank(tropicalized_matrix)
    results.append({
        "n": n,
        "size_P": size_P,
        "rank_value": rank_value
    })

metric_value = sum(result["rank_value"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(
    abs(math.log(result["size_P"]) - result["rank_value"]) <= 3 and result["rank_value"] >= 0
    for result in results
)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Minimal Rank of Tropicalized Geometric Quantization",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)

```

```

std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00bd4078.py", line 91, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00bd4078.py", line 62, in run_trial
    tropicalized_matrix = [[nodes[i][j] ^ nodes[j][i] for j in range(size_P)] for i in range(size_P)]
                             ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13560
2	preregistration	ollama_remote	glm4:latest	0	0	9767
3	novelty	ollama_remote	glm4:latest	0	0	8273
4	novelty	ollama_remote	glm4:latest	0	0	14410
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16275
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7504

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9213
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10950
9	judge	ollama_remote	glm4:latest	0	0	11669

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101622 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c1ce4a4a2e2e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c1ce4a4a2e2e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c1ce4a4a2e2e.tar.gz (if generated)